

OSSP Practical Experiments

Arun Tej (2520030472)

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        printf("Command execution failed\n");
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed\n");
    }

    return 0;
}
```

```

aruntej@LAPTOP-DGRT8C5G:~$ nano experiment-1.c
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-1.c -o experiment-1
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-1
Enter a Linux command: ls
Parent Process
Parent PID: 757
Child PID: 763
Child Process
Child PID: 763
Parent PID: 757
Hello.c  exp2.c  experiment  experiment.c  input.txt  output.txt  program
exp1.c  exp3    experiment-1  fork          notes.txt  program1    program
exp2    exp3.c  experiment-1.c  fork.c        ossp       program1.c  program
Child process completed
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-1.c -o experiment-1
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-1
Enter a Linux command: whoami
Parent Process
Parent PID: 789
Child PID: 790
Child Process
Child PID: 790
Parent PID: 789
aruntej
Child process completed
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-1.c -o experiment-1
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-1
Enter a Linux command: pwd
Parent Process
Parent PID: 818
Child PID: 821
Child Process
Child PID: 821
Parent PID: 818
/home/aruntej
Child process completed

```

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining

how the OS abstracts CPU, memory, storage, and I/O devices

```
aruntej@LAPTOP-DGRT8C5G:~$ nano experiment-1.c
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-1.c -o experiment-1
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-1
Enter a Linux command: ls
Parent Process
Parent PID: 757
Child PID: 763
Child Process
Child PID: 763
Parent PID: 757
Hello.c  exp2.c  experiment  experiment.c  input.txt  output.txt  program
exp1.c  exp3    experiment-1  fork          notes.txt  program1    program
exp2    exp3.c  experiment-1.c  fork.c        ossp       program1.c  program
Child process completed
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-1.c -o experiment-1
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-1
Enter a Linux command: whoami
Parent Process
Parent PID: 789
Child PID: 790
Child Process
Child PID: 790
Parent PID: 789
aruntej
Child process completed
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-1.c -o experiment-1
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-1
Enter a Linux command: pwd
Parent Process
Parent PID: 818
Child PID: 821
Child Process
Child PID: 821
Parent PID: 818
/home/aruntej
Child process completed
```

```

top - 13:14:20 up 13 min,  1 user,  load average: 0.00, 0.01, 0.00
Tasks:  26 total,   1 running, 25 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  7755.7 total,   7018.0 free,   583.1 used,   306.3 buff/cache
MiB Swap:  2048.0 total,   2048.0 free,    0.0 used.  7172.6 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24484	15436	11528	S	0.0	0.2	0:00.51	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.00	init
6	root	20	0	3180	2048	1940	S	0.0	0.0	0:00.00	init
43	root	19	-1	50428	16740	15476	S	0.0	0.2	0:00.13	systemd
78	systemd+	20	0	22416	14472	11976	S	0.0	0.2	0:00.05	systemd
85	root	20	0	35424	12332	9188	S	0.0	0.2	0:00.35	systemd
104	root	20	0	2888	1952	1820	S	0.0	0.0	0:00.04	chrony
105	root	20	0	4472	3084	2840	S	0.0	0.0	0:00.00	cron
106	message+	20	0	8824	5460	4780	S	0.0	0.1	0:00.04	dbus-
118	root	20	0	44096	29804	14668	S	0.0	0.4	0:00.29	networ
157	root	20	0	18436	9416	8176	S	0.0	0.1	0:00.09	systemd
185	syslog	20	0	220548	5868	4664	S	0.0	0.1	0:00.05	rsyslo
225	_chrony	20	0	20424	6464	5688	S	0.0	0.1	0:00.04	chrony
235	_chrony	20	0	12096	2440	1796	S	0.0	0.0	0:00.00	chrony
253	root	20	0	123460	32776	18164	S	0.0	0.4	0:00.15	unatt
287	root	20	0	5492	2660	2476	S	0.0	0.0	0:00.00	agetty
318	root	20	0	8648	5540	4696	S	0.0	0.1	0:00.00	login
375	aruntej	20	0	22336	13592	10980	S	0.0	0.2	0:00.07	systemd
377	aruntej	20	0	22916	4120	2116	S	0.0	0.1	0:00.00	(sd-pa
415	aruntej	20	0	6136	5412	3864	S	0.0	0.1	0:00.02	bash
695	root	20	0	3184	1116	980	S	0.0	0.0	0:00.00	Sessio
696	root	20	0	3200	1256	1108	S	0.0	0.0	0:00.01	Relay
697	aruntej	20	0	6268	5532	3864	S	0.0	0.1	0:00.05	bash
984	aruntej	20	0	8364	6140	3900	R	0.0	0.1	0:00.00	top
985	root	20	0	35428	6520	3340	S	0.0	0.1	0:00.00	(udev-
986	root	20	0	35428	6520	3340	S	0.0	0.1	0:00.00	(udev-

- Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

```
aruntej@LAPTOP-DGRT8C5G:~$ echo "Hello, this is a sample -
aruntej@LAPTOP-DGRT8C5G:~$ cat source.txt
Hello, this is a sample file.
aruntej@LAPTOP-DGRT8C5G:~$ nano experiment-2.c
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-2.c -o experiment-2
aruntej@LAPTOP-DGRT8C5G:~$ ./expreiment-2
-bash: ./expreiment-2: No such file or directory
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-2
File copied successfully.
aruntej@LAPTOP-DGRT8C5G:~$ cat destination.txt
Hello, this is a sample file.
```

Use the strace utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```

aruntej@LAPTOP-DGRT8C5G:~$ echo "OSSP Practical Experiment - 2" > sample.txt
aruntej@LAPTOP-DGRT8C5G:~$ cat sample.txt
OSSP Practical Experiment - 2
aruntej@LAPTOP-DGRT8C5G:~$ strace cat sample.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7fffea0b9dc8 /* 25 vars */) = 0
fork(NULL)                                = 0x60d795bdc000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7
access("/etc/ld.so.preload", R_OK)        = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20763, ...}) = 0
mmap(NULL, 20763, PROT_READ, MAP_PRIVATE, 3, 0) = 0x71129210c000
close(3)                                  = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEX
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 8
fstat(3, {st_mode=S_IFREG|0644, st_size=199120, ...}) = 0
mmap(NULL, 206288, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7112920d9000
mmap(0x7112920e0000, 135168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DE
mmap(0x711292101000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
mmap(0x711292108000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DEN
mmap(0x71129210a000, 5584, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANO
close(3)                                  = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXE
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 8
fstat(3, {st_mode=S_IFREG|0644, st_size=187120, ...}) = 0
mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7112920ab000
mmap(0x7112920af000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DE
mmap(0x7112920d3000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
mmap(0x7112920d7000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DEN
close(3)                                  = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) =
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 8
fstat(3, {st_mode=S_IFREG|0644, st_size=1198376, ...}) = 0
mmap(NULL, 1200152, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x711291f85000
mmap(0x711291f97000, 655360, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DE
mmap(0x711292037000, 466944, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
mmap(0x7112920a9000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DEN
close(3)                                  = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) =
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 8
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"...,
fstat(3, {st_mode=S_IFREG|0755, st_size=2186512, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"...,
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x711291c00000
mmap(0x711291c28000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_D
mmap(0x711291dc0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
mmap(0x711291e0e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DE
mmap(0x711291e14000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_AN
close(3)                                  = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEX
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 8
fstat(3, {st_mode=S_IFREG|0644, st_size=699056, ...}) = 0
mmap(NULL, 701112, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x711291ed9000
mmap(0x711291edc000, 507904, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DE
mmap(0x711291f58000, 176128, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
mmap(0x711291f83000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DEN

```



```

close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libpcr2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 8) = 16
fstat(3, {st_mode=S_IFREG|0644, st_size=699056, ...}) = 0
mmap(NULL, 701112, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x711291ed9000
mmap(0x711291edc000, 507904, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0x711291edc000
mmap(0x711291f58000, 176128, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0x711291f58000
mmap(0x711291f83000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0x711291f83000
close(3) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x711291f83000
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x711291f83000
arch_prctl(ARCH_SET_FS, 0x711291ed4b00) = 0
set_tid_address(0x711291ed5128) = 2072
set_robust_list(0x711291ed4de0, 24) = 0
rseq(0x711291ed46a0, 0x20, 0, 0x53053053) = 0
mprotect(0x711291e0e000, 16384, PROT_READ) = 0
mprotect(0x711291f83000, 4096, PROT_READ) = 0
mprotect(0x7112920a9000, 4096, PROT_READ) = 0
mprotect(0x7112920d7000, 4096, PROT_READ) = 0
mprotect(0x711292108000, 4096, PROT_READ) = 0
mprotect(0x60d763563000, 1462272, PROT_READ) = 0
mprotect(0x711292157000, 8192, PROT_READ) = 0
rlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\x31\xa2\x4d\x82\xd3\x6d\x8b\x6d", 8, GRND_NONBLOCK) = 8
munmap(0x71129210c000, 20763) = 0
statfs("/sys/fs/selinux", {f_type=SYSFS_MAGIC, f_bsize=4096, f_blocks=0, f_bfrsize=4096, f_files=0, f_namelen=255}) = 0
statfs("/selinux", 0x7fff60124230) = -1 ENOENT (No such file or directory)
brk(NULL) = 0x60d795bdc000
brk(0x60d795bfd000) = 0x60d795bfd000
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tbdev\nnodev\tfd\nnodev\tfuse\nnodev\t"..., 1024) = 442
lseek(3, 0, SEEK_CUR) = 442
lseek(3, 416, SEEK_SET) = 416
close(3) = 0
openat(AT_FDCWD, "/proc/mounts", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "/dev/sdd / ext4 rw,relatime,disc"..., 1024) = 1024
read(3, " tmpfs rw,nosuid,nodev,mode=755 "..., 1024) = 1024
read(3, ",upperdir=/gpu_lib/rw/upper,work"..., 1024) = 1024
read(3, "th=C:\\;uid=1000;gid=1000;symlink"..., 1024) = 299
read(3, "", 1024) = 0
close(3) = 0
access("/etc/selinux/config", F_OK) = -1 ENOENT (No such file or directory)
fcntl(0, F_GETFD) = 0
fcntl(1, F_GETFD) = 0
fcntl(2, F_GETFD) = 0
rt_sigaction(SIGPIPE, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
poll([{fd=0, events=0}, {fd=1, events=0}, {fd=2, events=0}], 3, 0) = 0 (Timeout)
rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[PIPE], sa_flags=SA_RESTORESIGPIPE}, 8) = 0
openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
rlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "60d762bfa000-60d762cec000 r--p 0"..., 1024) = 1024
read(3, "0 08:30 13647"..., 1024) = 1024

```


+++ exited with 0 +++

```
aruntej@LAPTOP-DGRT8C5G:~$ strace -e trace=openat,read,write,close cat sample
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 8192) = 8192
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 8192) = 8192
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 8192) = 8192
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 8192) = 8192
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 8192) = 8192
close(3) = 0
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tbd"..., 1024) = 442
close(3) = 0
openat(AT_FDCWD, "/proc/mounts", O_RDONLY|O_CLOEXEC) = 3
read(3, "/dev/sdd / ext4 rw,relatime,disc"..., 1024) = 1024
read(3, " tmpfs rw,nosuid,nodev,mode=755 "..., 1024) = 1024
read(3, ",upperdir=/gpu_lib/rw/upper,work"..., 1024) = 1024
read(3, "th=C:\\;uid=1000;gid=1000;symlink"..., 1024) = 299
read(3, "", 1024) = 0
close(3) = 0
openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
read(3, "5639fd74f000-5639fd841000 r--p 0"..., 1024) = 1024
read(3, "0 08:30 13647"..., 1024) = 1024
read(3, "00 08:30 13650"..., 1024) = 1024
read(3, " /usr/lib/x86_64-lin"..., 1024) = 1024
read(3, "0 13644 /us"..., 1024) = 566
close(3) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/uucore/en-US.ftl", O_RDONLY|O_CLOEXEC) = 3
read(3, "# Common strings shared across a"..., 4080) = 4080
read(3, "", 32) = 0
close(3) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/cat/en-US.ftl", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "sample.txt", O_RDONLY|O_CLOEXEC) = 3
OSSP Practical Experiment - 2
close(5) = 0
close(4) = 0
close(3) = 0
+++ exited with 0 +++
aruntej@LAPTOP-DGRT8C5G:~$ |
```

3. Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

```

GNU nano 0.7.1
int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        printf("Child is going to WAIT/SLEEP...\n");
        sleep(10);

        printf("Child is now RUNNING...\n");

        // CPU intensive work
        for (long long i = 0; i < 5000000000; i++)
        {
            // Busy work
        }

        printf("Child is TERMINATING...\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("\n--- PARENT PROCESS ---\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        printf("Child PID : %d\n", pid);

        printf("Parent is WAITING for child...\n");

        wait(NULL);

        printf("Child has TERMINATED.\n");
        printf("Parent is TERMINATING...\n");
    }

    return 0;
}

```

```

aruntej@LAPTOP-DGRT8C5G:~$ nano experiment-3.c
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-3.c -o experiment-3
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-3

--- PARENT PROCESS ---
PID      : 2137
PPID     : 1774
Child PID : 2138
Parent is WAITING for child...

--- CHILD PROCESS ---
PID : 2138
PPID : 2137
Child is going to WAIT/SLEEP...
Child is now RUNNING...
Child is TERMINATING...
Child has TERMINATED.
Parent is TERMINATING...

```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```

aruntej@LAPTOP-DGRT8C5G:~$ nano experiment-4.c
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-4.c -o experiment-4
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-4
Parent Process: PID = 2283
Child 1: PID = 2284, PPID = 2283

Parent using wait():
Child 2: PID = 2285, PPID = 2283
Child 3: PID = 2286, PPID = 2283
Child 1: PID = 2284 completed
wait(): Child PID 2284 terminated
Child 2: PID = 2285 completed
wait(): Child PID 2285 terminated

Parent using waitpid():
Child 3: PID = 2286 completed
waitpid(): Child PID 2286 terminated

All child processes completed.
aruntej@LAPTOP-DGRT8C5G:~$ |

```

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques

```
aruntej@LAPTOP-DGRT8C5G:~$ nano zombie.c
aruntej@LAPTOP-DGRT8C5G:~$ gcc zombie.c -o zombie
aruntej@LAPTOP-DGRT8C5G:~$ ./zombie
Parent: PID = 2325
Child PID = 2326
Parent is sleeping...
Child: PID = 2326
Child is terminating...
Parent exiting...
aruntej@LAPTOP-DGRT8C5G:~$ |
```

```
aruntej@LAPTOP-DGRT8C5G:~$ ps -ef | grep zombie
aruntej      2366      2331  0 16:17 pts/2    00:00:00 grep
aruntej@LAPTOP-DGRT8C5G:~$ nano zombie_fixed.c
aruntej@LAPTOP-DGRT8C5G:~$ gcc zombie_fixed.c -o zombie_f
aruntej@LAPTOP-DGRT8C5G:~$ ./zombie_fixed
Parent: PID = 2400
Child PID = 2401
Child: PID = 2401
Child is terminating...
Parent collected child process.
No zombie process remains.
aruntej@LAPTOP-DGRT8C5G:~$ █
```


